

3. GraphQL Implementation

Step by step untuk implementasi GraphQL di project **HR-Service-API** :

1. Install Dependencies.
2. Create TypeDefs (Schema Definition Language).
3. Buat mappers (mapping fields camelCase <-> snake_case)
4. Create error-mapper.
5. Create Loaders (DataLoader Batching).
6. Create Mercurius-Context (Module Augmentation)
7. Create Resolver, dan Connect to Service.
8. Create Schema.
9. Wiring to Apps.ts

1. Install Dependencies

- Kita gunakan mercurius (alternatif dari apollo, tapi mercurius udah plug & play dengan otel) plugin bawaan fastify.

```
npm install mercurius graphql dataloader
```

- Install code generator untuk build resolver.

```
npm install -D @graphql-codegen/cli @graphql-codegen/typescript @graphql-codegen/typescript-resolvers
```

2. Create TypeDefs (Schema Definition Language)

File: **src/modules/employees/graphql/employees.typeDefs.ts**. First foundation — berisi string SDL (type Employee, Query, Mutation, dst), tidak bergantung ke file GraphQL lain. Tulis field mengikuti kolom employees.schema.ts (TypeBox) yang sudah ada, dengan gaya camelCase.

```
export const employeeTypeDefs = `
  enum EmploymentStatus {
    ACTIVE
    ON_LEAVE
    SUSPENDED
    RESIGNED
    TERMINATED
  }

  enum EmploymentType {
    PERMANENT
    CONTRACT
  }
`
```

```
    INTERN
    PROBATION
  }

  type Job {
    jobId: Int!
    jobTitle: String!
  }

  type Department {
    departmentId: Int!
    departmentName: String!
    locationId: Int
  }

  type Employee {
    employeeId: Int!
    firstName: String
    lastName: String!
    email: String!
    phoneNumber: String
    hireDate: String!
    jobId: Int!
    job: Job
    salary: Float!
    managerId: Int
    manager: Employee
    directReports: [Employee!]!
    departmentId: Int
    department: Department
    employmentStatus: EmploymentStatus!
    employmentType: EmploymentType!
    terminationDate: String
    createdAt: String!
    updatedAt: String!
  }

  type Pagination {
    page: Int!
    limit: Int!
    total: Int!
    totalPages: Int!
  }

  type EmployeeConnection {
    items: [Employee!]!
    pagination: Pagination!
  }

  input EmployeeFilterInput {
    search: String
    departmentId: Int
    jobId: Int
    managerId: Int
```

```

    employmentStatus: EmploymentStatus
    employmentType: EmploymentType
  }

  input CreateEmployeeInput {
    firstName: String!
    lastName: String!
    email: String!
    phoneNumber: String!
    hireDate: String!
    jobId: Int!
    salary: Float!
    managerId: Int
    departmentId: Int
    employmentStatus: EmploymentStatus
    employmentType: EmploymentType
    terminationDate: String
  }

  input UpdateEmployeeInput {
    firstName: String!
    lastName: String!
    email: String!
    phoneNumber: String!
    hireDate: String!
    jobId: Int!
    salary: Float!
    managerId: Int
    departmentId: Int
    employmentStatus: EmploymentStatus
    employmentType: EmploymentType
    terminationDate: String
  }

  type Query {
    employees(page: Int = 1, limit: Int = 20, filter: EmployeeFilterInput):
EmployeeConnection!
    employee(id: Int!): Employee
  }

  type Mutation {
    createEmployee(input: CreateEmployeeInput!): Employee!
    updateEmployee(id: Int!, input: UpdateEmployeeInput!): Employee!
    deleteEmployee(id: Int!): Boolean!
  }

```

2. Buat mappers (camelCase <-> snake_case)

File: **src/modules/employees/graphql/employees.mappers.ts**. Bergantung ke **employees.schema.ts** (CreateEmployeeInput/UpdateEmployeeInput) yang sudah ada. Isinya cuma 2 fungsi murni pemetaan nama field, tidak ada logic bisnis.

```

import type {
  CreateEmployeeInput as GqlCreateEmployeeInput,
  UpdateEmployeeInput as GqlUpdateEmployeeInput,
} from '../..../graphql/generated/resolvers-types'
import type { CreateEmployeeInput, UpdateEmployeeInput } from
'../employees.schema'

export function mapCreateInput(input: GqlCreateEmployeeInput): CreateEmployeeInput
{
  return {
    first_name: input.firstName,
    last_name: input.lastName,
    email: input.email,
    phone_number: input.phoneNumber,
    hire_date: input.hireDate,
    job_id: input.jobId,
    salary: input.salary,
    manager_id: input.managerId,
    department_id: input.departmentId,
    employment_status: input.employmentStatus as
CreateEmployeeInput['employment_status'],
    employment_type: input.employmentType as
CreateEmployeeInput['employment_type'],
    termination_date: input.terminationDate,
  }
}

export function mapUpdateInput(input: GqlUpdateEmployeeInput): UpdateEmployeeInput
{
  return {
    first_name: input.firstName,
    last_name: input.lastName ?? undefined,
    email: input.email ?? undefined,
    phone_number: input.phoneNumber,
    hire_date: input.hireDate ?? undefined,
    job_id: input.jobId ?? undefined,
    salary: input.salary ?? 0.0,
    manager_id: input.managerId,
    department_id: input.departmentId,
    employment_status: input.employmentStatus as
UpdateEmployeeInput['employment_status'],
    employment_type: input.employmentType as
UpdateEmployeeInput['employment_type'],
    termination_date: input.terminationDate,
  }
}

```

3.Create error-mapper

File: **src/common/graphql/error-mapper.ts**. Bergantung ke **api-error.ts** yang sudah ada (statusCode, message). Fungsinya mengubah ApiError jadi GraphQLError dengan extensions.code. Independen dari file

GraphQL lain, bisa dibuat kapan saja setelah step 1.

```
// GraphQL: semua response tetap HTTP 200; error masuk ke `errors[]` dengan
// `extensions.code`. File ini jembatannya, ApiError.ts sendiri TIDAK diubah.

import { GraphQLError } from 'graphql'
import { ApiError } from '../utils/api-error'

const statusToCode: Record<number, string> = {
  400: 'BAD_REQUEST',
  404: 'NOT_FOUND',
  409: 'CONFLICT',
}

export function toGraphQLError(err: unknown): GraphQLError {
  if (err instanceof ApiError) {
    return new GraphQLError(err.message, {
      extensions: { code: statusToCode[err.statusCode] ?? 'INTERNAL_SERVER_ERROR'
    },
  })
  }
  if (err instanceof GraphQLError) return err

  // Jangan bocorkan detail error asli (misal SQL error) ke client.
  return new GraphQLError('Internal server error', {
    extensions: { code: 'INTERNAL_SERVER_ERROR' },
  })
}
```

4.Create loaders (DataLoader batching)

File: **src/modules/employees/graphql/employees.loaders.ts**. Bergantung ke db/schema.ts (tabel jobs, departments, employees) yang sudah ada. Isinya 4 DataLoader: jobLoader, departmentLoader, employeeLoader, directReportsLoader.

```
import DataLoader from 'dataloader'
import { inArray } from 'drizzle-orm'
import { db } from '../../../db'
import { jobs, departments, employees } from '../../../db/schema'

// drizzle saat baca tipe data numeric, akan ubah ke string
// karena itu ubah tipe data salary dari numeric ke number
type EmployeeRow = typeof employees.$inferSelect
type EmployeeApiRow = Omit<EmployeeRow, 'salary'> & { salary: number }

function toApiRow(row: EmployeeRow): EmployeeApiRow {
  return { ...row, salary: Number(row.salary) }
}

export interface EmployeeLoaders {
```

```

jobLoader: DataLoader<number, typeof jobs.$inferSelect | null>
departmentLoader: DataLoader<number, typeof departments.$inferSelect | null>
employeeLoader: DataLoader<number, EmployeeApiRow | null>
directReportsLoader: DataLoader<number, EmployeeApiRow[]>
}

export function createEmployeeLoaders(): EmployeeLoaders {
  const jobLoader = new DataLoader<number, typeof jobs.$inferSelect | null>(async
(ids) => {
    const rows = await db.select().from(jobs).where(inArray(jobs.jobId, [...ids]))
    const byId = new Map(rows.map((r) => [r.jobId, r]))
    return ids.map((id) => byId.get(id) ?? null)
  })

  const departmentLoader = new DataLoader<number, typeof departments.$inferSelect
| null>(async (ids) => {
    const rows = await
db.select().from(departments).where(inArray(departments.departmentId, [...ids]))
    const byId = new Map(rows.map((r) => [r.departmentId, r]))
    return ids.map((id) => byId.get(id) ?? null)
  })

  // Dipakai untuk resolve `manager` (self-referencing FK).
  const employeeLoader = new DataLoader<number, EmployeeApiRow | null>(async (ids)
=> {
    const rows = await
db.select().from(employees).where(inArray(employees.employeeId, [...ids]))
    const byId = new Map(rows.map((r) => [r.employeeId, toApiRow(r)]))
    return ids.map((id) => byId.get(id) ?? null)
  })

  // `directReports`: kebalikan arah dari manager.
  const directReportsLoader = new DataLoader<number, EmployeeApiRow[]>(async
(managerIds) => {
    const rows = await
db.select().from(employees).where(inArray(employees.managerId, [...managerIds]))
    return managerIds.map((mid) => rows.filter((r) => r.managerId ===
mid).map(toApiRow))
  })

  return { jobLoader, departmentLoader, employeeLoader, directReportsLoader }
}

```

5. Create mercurius-context (module augmentation)

File: **src/graphql/mercurius-context.ts**. Bergantung ke `employees.loaders.ts` (step 4) untuk import tipe `EmployeeLoaders`. Isinya cuma declare module 'mercurius' supaya `MercuriusContext` resmi punya field `loaders`.

```

import type { EmployeeLoaders } from
'../modules/employees/graphql/employees.loaders'

```

```
declare module 'mercurius' {
  interface MercuriusContext {
    loaders: EmployeeLoaders
  }
}

// export {} supaya file ini dianggap module oleh TS,
export {}
```

6.Create resolvers, sambungkan ke service

File: **src/modules/employees/graphql/employees.resolvers.ts**. Titik temu SEMUA step sebelumnya (typedefs, mappers, loaders, error-mapper) plus **employees.service.ts** yang sudah ada. Resolver di sini TIDAK boleh punya business logic sendiri, cuma manggil service.

```
import * as employeeService from '../employees.service'
import { mapCreateInput, mapUpdateInput } from './employees.mappers'
import { toGraphQLError } from '../../../common/graphql/error-mapper'
import type { Resolvers } from '../../../graphql/generated/resolvers-types'
import type { ListEmployeeQuery } from '../employees.schema'

export const employeeResolvers: Resolvers = {
  Query: {
    employees: async (_parent, args) => {
      try {
        const { items, pagination } = await employeeService.listEmployees({
          page: args.page,
          limit: args.limit,
          ...args.filter,
        }) as ListEmployeeQuery
        return { items, pagination }
      } catch (err) {
        throw toGraphQLError(err)
      }
    },
  },
  employee: async (_parent, args) => {
    try {
      return await employeeService.getEmployeeById(args.id)
    } catch (err) {
      // ApiError.notFound -> GraphQLError { extensions: { code: 'NOT_FOUND' } }
      throw toGraphQLError(err)
    }
  },
},

Mutation: {
  createEmployee: async (_parent, args) => {
    try {
      return await employeeService.createEmployee(mapCreateInput(args.input))
    }
  },
}
```

```

    } catch (err) {
      throw toGraphQLError(err)
    }
  },

  updateEmployee: async (_parent, args) => {
    try {
      return await employeeService.updateEmployee(args.id,
mapUpdateInput(args.input))
    } catch (err) {
      throw toGraphQLError(err)
    }
  },

  deleteEmployee: async (_parent, args) => {
    try {
      await employeeService.deleteEmployee(args.id)
      return true
    } catch (err) {
      throw toGraphQLError(err)
    }
  },
},

Employee: {
  job: (parent, _args, ctx) => ctx.loaders.jobLoader.load(parent.jobId),

  department: (parent, _args, ctx) =>
    parent.departmentId ? ctx.loaders.departmentLoader.load(parent.departmentId)
: null,

  manager: (parent, _args, ctx) =>
    parent.managerId ? ctx.loaders.employeeLoader.load(parent.managerId) : null,

  directReports: (parent, _args, ctx) =>
    ctx.loaders.directReportsLoader.load(parent.employeeId),
},
}

```

7.Create root schema.ts (merge)

File: **src/graphql/schema.ts**. Bergantung ke typedefs (step 1) dan resolvers (step 6). Untuk sekarang cuma re-export employeeTypeDefs & employeeResolvers, jadi satu titik import tunggal buat app.ts.

```

// Taruh di: src/graphql/schema.ts
//karena ada 1 module (employees), jadi cukup kita define employee saja.

import { employeeTypeDefs } from '../modules/employees/graphql/employees.typedefs'
import { employeeResolvers } from
'../modules/employees/graphql/employees.resolvers'

```



```
export const schema = employeeTypeDefs

export const resolvers = employeeResolvers
```

8. Wiring ke app.ts

Edit file yang sudah ada: src/app.ts. Bergantung ke schema.ts (step 7), employees.loaders.ts (step 4), dan mercurius-context.ts (step 5, WAJIB di-import walau tidak dipakai langsung supaya augmentation aktif). Tambah app.register(mercurius, {...}) dengan context factory yang panggil createEmployeeLoaders().

```
import cors from '@fastify/cors';
import helmet from '@fastify/helmet';
import fastifySwagger from '@fastify/swagger'
import fastifySwaggerUi from '@fastify/swagger-ui'

import multipart from '@fastify/multipart'
import { MAX_FILE_SIZE_BYTES } from '../modules/employee-files/employee-files.schema'

import { TypeBoxTypeProvider } from '@fastify/type-provider-typebox'
import Fastify from 'fastify';
import { errorHandler, notFoundHandler } from '../common/handlers/error-handler';
import { env } from '../config/env';
import { apiRoutes } from '../routes';

import mercurius from 'mercurius'
import { schema, resolvers } from '../graphql/schema'
import { createEmployeeLoaders } from
'../modules/employees/graphql/employees.loaders'
import '../graphql/mercurius-context' //

export async function createApp() {

  const app = Fastify({
    logger: env.NODE_ENV === 'test'
      ? false
      : {
        level: env.NODE_ENV === 'production' ? 'info' : 'debug',
        transport: {
          targets: [
            {
              // we keep pino-log for locale development only
              target: 'pino-pretty',
              level: 'debug',
              options: { colorize: true, translateTime: 'HH:MM:ss', ignore:
'pid,hostname' },
            },
            {
              // Kirim log yang sama ke Loki lewat OTLP. Trace_id/span_id
```

```

        // sudah otomatis ke-inject oleh PinoInstrumentation di
        // instrumentation.ts, jadi log ini bisa langsung dikorelasikan
        // ke trace yang sesuai di Jaeger.
        target: 'pino-opentelemetry-transport',
        level: 'info', // tidak perlu kirim 'debug' verbose ke Loki
        options: {
            loggerName: process.env.OTEL_SERVICE_NAME ?? 'hr-service-api',
            serviceVersion: '1.0.0',
        },
    },
],
},
},
}).withTypeProvider<TypeBoxTypeProvider>();

await app.register(fastifySwagger, {
  openapi: {
    info: {
      title: 'HR Service API',
      description: 'API dokumentasi untuk HR microservice',
      version: '1.0.0',
    },
    servers: [
      { url: 'http://localhost:3002', description: 'Development' },
    ],
  },
})

await app.register(fastifySwaggerUi, {
  routePrefix: '/documentation',
})

await app.register(helmet);
await app.register(cors);

// Body parsing JSON sudah bawaan Fastify (`content-type: application/json`),
// tidak perlu plugin/middleware tambahan seperti `express.json()`.

//await app.register(employeeFileRoutes, { prefix:
'/employees/:employeeId/files' })

app.setNotFoundHandler(notFoundHandler);
app.setErrorHandler(errorHandler);

app.get('/health', async () => ({ success: true, message: 'hr-service-api is
healthy' }));

await app.register(multipart, { limits: { fileSize: MAX_FILE_SIZE_BYTES } })
await app.register(apiRoutes, { prefix: env.API_PREFIX });

await app.register(mercurius, {

```

```
    schema,  
    resolvers,  
    graphiql: env.NODE_ENV !== 'production',  
    context: async () => ({  
      loaders: createEmployeeLoaders(), // wajib dibuat ULANG tiap request, jangan  
di-hoist ke luar  
    }),  
  });  
  
  return app;  
}
```